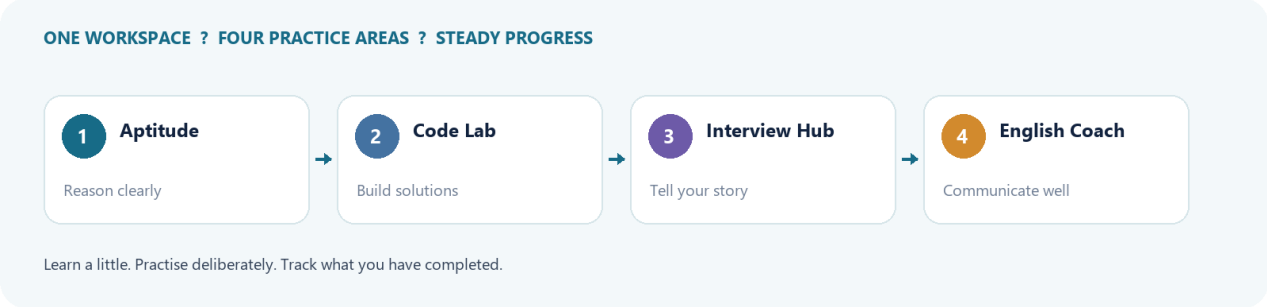
LEARNSPHERE AI

A Project Overview

A practical learning and placement preparation platform

PROJECT BRIEF | PRODUCT, FEATURES, ARCHITECTURE & OPERATIONS



LearnSphere AI brings interview preparation, aptitude practice, coding, English communication, and AI-guided study into one web application. This document describes the current project structure and the learner experience represented in the repository.

Prepared from the application source and templates ? 25 October 2026

01 / THE PROJECT

Executive summary

LearnSphere AI is a Flask-based web application for learners preparing for internships, entry-level roles, and placement interviews. Its central idea is to make preparation continuous: learners can practise, receive guidance, and track selected progress in one place.

Purpose

- Connect technical and non-technical preparation in a single learner workspace.
- Support repeatable practice rather than one-time content consumption.
- Use AI assistance for explanations, feedback, and generated practice where configured.

Intended audience

Students and early-career applicants preparing for aptitude tests, coding screens, interviews, and workplace communication.

Product approach

The application combines structured practice pages with a dashboard and saved activity. Some tasks are generated dynamically through Gemini; other learning content and plans are rule-based or curated in the application.

02 / PRODUCT EXPERIENCE

Learner experience and dashboard

After creating an account and signing in, a learner enters a dashboard that acts as a launchpad for the preparation tools. Authentication protects the learning workspace, and progress indicators connect practice activity to the wider placement goal.

Main navigation

The current templates expose the core areas: Aptitude Arena, Code Lab, Interview Hub, English Coach, Study Assistant, AI Coach, and Placement Plan.

Progress signals

- Quiz attempts record topic, question count, correct answers, and percentage.
- Interview challenge completion is saved by learner and level.
- English practice tracks a daily practice date and streak.
- Placement-plan tasks can be marked complete for a given date.

Design principle

Make the next action obvious: practise a topic, continue a level, ask for help, or complete a daily preparation task. Progress measures should encourage consistency without claiming job-readiness from a single score.

03 / LEARNING TOOLS

Aptitude Arena

Aptitude Arena supports practice in areas commonly assessed in hiring tests. The application defines guides and tips for quantitative aptitude, logical reasoning, Python programming, operating systems, database systems, and computer networks.

Learning loop

- Choose a topic and work through generated or available questions.
- Submit answers and review correctness and performance.
- Save quiz results so the dashboard can show a record of practice.

Coaching cues

Topic guides offer practical strategies, such as estimating a range before calculating, writing logic constraints down, checking edge cases, and explaining reasoning aloud.

Value for interview preparation

Aptitude practice builds speed and accuracy while the included interview tips prompt learners to communicate their reasoning, not just produce an answer.

04 / LEARNING TOOLS

Code Lab

Code Lab provides coding practice and challenge generation in a dedicated part of the platform. Its service module integrates with Gemini to create coding challenges when the API is configured.

Practice activities

- Review a coding prompt and work on a solution.
- Run submitted code through the application?s runner route.
- Record a completed coding challenge where the page flow supports it.

Learning habits

Start with a small example, reason about the data structure, test boundary cases, and explain time and space complexity after the solution works.

Operational consideration

Executing learner-provided code is inherently sensitive. The current local runner should be treated as a learning prototype and must not be exposed to untrusted users without process isolation, resource limits, timeouts, and a hardened execution boundary.

05 / LEARNING TOOLS



Interview Hub and 100 levels

Interview Hub turns behavioural and communication practice into a sequence of 100 levels. The level prompts cycle through a set of core topics and vary the coaching focus as learners progress.

Prompt themes

- Tell me about yourself and role motivation.
- Project deep dives and technical explanations.
- Problem solving, ownership, setbacks, and reflection.
- Teamwork, conflict, and feedback.

Completion and progress

Each level can be submitted as a written response. Completions are stored per user and level, preventing duplicate completion records for the same level. The dashboard can use these saved records to show progress.

Practice method

Use a clear situation, describe your own actions, state the result, and finish with a learning point. Practise concise answers, then revise for specificity and a natural speaking voice.

06 / LEARNING TOOLS

English Coach

English Coach focuses on practical communication for interviews and the workplace. Its service code calls Gemini-backed feedback functions for written English and pronunciation practice feedback.

Practice modes

The application includes a dedicated English Coach page and a pronunciation feedback route. The learner can submit practice and receive coaching feedback when the AI service is available.

Daily consistency

The account model stores the last English practice date and a streak counter. The streak increments for consecutive days and starts again after a missed day.

Suggested routine

- Write or speak a short response to a realistic interview question.
- Review feedback and select one point to improve.
- Repeat the response with clearer wording and stronger examples.

07 / LEARNING TOOLS

AI Coach and Study Assistant

The project provides two AI-assisted experiences: an AI Coach response flow and a Study Assistant chat flow. Both are intended to make help available while a learner is studying rather than forcing them to leave the platform.

Study Assistant

Learner messages are sent to a service module that calls Gemini. Message records include the user, role, study mode, content, and timestamp.

AI Coach

The AI Coach has its own page and response endpoint. It complements structured modules with contextual help for questions that arise during preparation.

Effective use

- Ask for a concept explanation at your current level.
- Request a worked example, then solve a similar example yourself.
- Ask for feedback on an answer and revise it rather than copying it verbatim.

Availability

AI features depend on a valid API key, a supported model name, network availability, and provider limits. The app includes an availability error message for the Study Assistant.

08 / LEARNING TOOLS

Placement Plan

The Placement Plan organizes preparation into a seven-day rotation. Each day pairs activities across aptitude, coding, interviews, and English so learners build a balanced routine.

Plan structure

The weekly schedule rotates quantitative aptitude, logical reasoning, Python, DBMS, and computer networks; arrays, strings, and algorithms; interview themes; and English conversation, grammar, vocabulary, and interview practice.

Task completion

A learner can mark plan tasks complete. Completions are saved against the user, plan date, and task key, with a uniqueness constraint to avoid duplicate daily task entries.

How to use it

- Choose a realistic daily time budget before starting.
- Complete one focused block per skill area rather than multitasking.
- Use missed days as a signal to adjust workload, not abandon the plan.

09 / ACCOUNT & SECURITY

Accounts, sign-in, and recovery

LearnSphere AI includes account registration, sign-in, sign-out, and password reset flows. New accounts validate required fields, email format, password length, confirmation, and email uniqueness.

Password storage

Passwords are stored as hashes using Werkzeug?s password hashing helpers; the raw password is not saved in the user table.

Password reset

Reset links are signed with itsdangerous and include the current password hash in the token payload. The configured reset age is one hour. Email delivery uses SMTP settings from environment variables.

Session access

Flask-Login manages user sessions and the login-required decorator protects authenticated views and submission endpoints.

Deployment hygiene

Set a strong SECRET_KEY, restrict email credentials to environment configuration, use HTTPS in production, and ensure error responses do not reveal account details.

10 / TECHNICAL DESIGN



Application architecture

The application is organized around a Flask entry point, configuration, SQLAlchemy models, service modules, templates, and static assets. The arrangement keeps route handling and user flows in the main application while delegating AI provider calls to focused services.

Major components

- app.py defines routes, authentication flows, data handling, and the learning pages.
- models.py defines SQLAlchemy records and database constraints.
- config.py reads secrets and mail configuration from environment variables and sets the SQLite database URI.
- services/ contains Gemini integrations for questions, coding challenges, English feedback, and study assistance.
- templates/ contains Jinja HTML views; static/ contains CSS assets.

Request lifecycle

A browser submits a form or request to a Flask route. The route checks session state and input, calls a service if needed, reads or writes database records, then returns a rendered page or JSON response.

Database initialization

The application initializes SQLAlchemy and calls create_all during startup within an application context. The local database is stored under instance/learnsphere.db.

11 / TECHNICAL DESIGN

Data model and progress records

The current schema stores account identity and several forms of learning progress. It is designed for a single-user learning workspace rather than a multi-tenant analytics platform.

Primary records

- User: name, email, password hash, role, and English practice streak fields.
- QuizAttempt: topic, total and correct questions, percentage, and creation time.
- InterviewChallengeCompletion: level, submitted response, completion time, and unique user-level pairing.
- InterviewPracticeSession: topic, prompt, response, word count, and timestamp.
- StudyAssistantMessage: role, mode, content, and timestamp.
- PlacementPlanCompletion: plan date, task key, completion time, and unique user-date-task pairing.

Data handling

User-owned records are linked by foreign keys. Routes should continue to scope queries to the authenticated user and validate all incoming values before persistence.

Future schema evolution

If deployed beyond a prototype, introduce Alembic migrations, explicit retention policies for chat and practice content, and indexes based on real query patterns.

12 / OPERATIONS

AI and API configuration

AI-backed services use the Google GenAI Python client. Configuration is loaded from the environment, including GEMINI_API_KEY and GEMINI_MODEL. The current default model string is defined in each service module unless overridden.

Configuration checklist

- Create a local .env file from the expected variable names.
- Set GEMINI_API_KEY to a valid key without committing the value.
- Set GEMINI_MODEL to a model available to the account.
- Restart the Flask process after environment changes.

Failure behavior

A missing key or provider error can prevent generation and feedback. Keep a graceful user-facing message, log enough diagnostic context for maintainers, and avoid exposing secrets or full provider responses to learners.

API key verification

A key is confirmed only when a real authenticated API request succeeds. Merely having a non-empty environment variable does not prove that the key is valid, enabled, or within quota.

Cost and privacy

Send only the information needed to generate feedback. Explain that AI output may be inaccurate, and avoid sending sensitive personal data or confidential employer material.

13 / OPERATIONS

Local setup and operation

A developer can run the application locally with Python and the dependencies installed in the project environment. The repository contains a venv directory, but fresh setup should use a clean environment and declared dependencies if a requirements file is available.

Typical startup

- Open a terminal in the project directory.
- Activate the project virtual environment or create one.
- Install the project dependencies.
- Configure environment values in .env.
- Run the Flask application using the project?s configured entry point.

Important environment variables

SECRET_KEY controls session and token signing. GEMINI_API_KEY and GEMINI_MODEL control Gemini calls. MAIL_SERVER, MAIL_PORT, MAIL_USE_TLS or MAIL_USE_SSL, MAIL_USERNAME, MAIL_PASSWORD, and MAIL_DEFAULT_SENDER support password-reset email.

Local persistence

SQLite data is stored in instance/learnsphere.db. Keep this file backed up if local learner history matters. Do not use create_all as a production migration strategy.

14 / QUALITY & ROADMAP

Quality, security, and accessibility

A polished learning product needs dependable behavior as well as useful features. Current implementation choices provide a practical starting point; production use calls for additional verification, privacy practices, and operational safeguards.

Recommended improvements

- Add automated tests for registration, authentication, reset tokens, ownership checks, and progress calculations.
- Add database migrations and a documented dependency lock.
- Protect code execution using isolated containers or a dedicated sandbox service.
- Add rate limiting and request size limits to AI and code routes.
- Provide accessible labels, keyboard focus states, readable contrast, and responsive layouts across all templates.
- Add structured logging and clear health checks without leaking credentials.

Product roadmap

- Personalized practice recommendations based on weak topics.
- Answer history with self-review notes and revision comparisons.
- Mock interview sessions with timed prompts and structured rubrics.
- Placement readiness dashboard with transparent component scores.
- Exportable study plan and reminders.

Success indicators

Track repeated practice, topic coverage, improvement over multiple attempts, completion of weekly plans, and learner-reported confidence. Avoid treating usage volume alone as evidence of learning.

15 / REFERENCE

Appendix: routes and repository map

The route list and repository map provide a quick orientation for maintainers. Exact behavior should be confirmed against the source before a deployment or integration change.

Main routes

- / and /dashboard provide the public landing and authenticated learner dashboard.
- /register, /login, /logout, /forgot-password, and /reset-password/<token> handle account access.
- /aptitude and /code-lab serve practice modules; /run-code handles code submissions.
- /interview-hub and /challenges/<level> support interview practice and level completion.
- /english-coach-page, /english-coach, and /pronunciation-feedback support English practice.
- /study-assistant and /study-assistant/respond support study chat; /ai-coach and /ai-coach/respond support coach interactions.
- /placement-plan and /placement-plan/complete/<task_key> handle the weekly plan.

Repository map

app.py ? Flask routes and workflows | models.py ? database records | config.py ? settings | services/ ? AI integrations | templates/ ? Jinja pages | static/ ? CSS and frontend assets | instance/ ? local SQLite data.

Project note

This overview describes the codebase at the time of preparation. It is a project brief for orientation and planning, not a guarantee that every third-party service or production safeguard has been configured.